

Chapter 10. Introduction to Observability

The topic of observability has come more and more to the forefront of software development in recent years.

Observability is making the transition from being a niche concern to becoming a new frontier for user experience, systems, and service management in web companies and enterprises alike.

—[James Governor](#)

But why should Java developers care about observability? And what the hell is observability anyway?

In this chapter, we will explore the concepts and fundamentals of observability, and in [Chapter 11](#), we will see how these techniques can be deployed in Java applications using open source libraries and technologies.

The What and the Why of Observability

Observability has a reputation, among some developers, of being vague and difficult to understand. This is, in our view, unwarranted—observability is conceptually simple and should be easy to explain. Observability tools are fundamentally a continuation, extension, and generalization of classic monitoring systems to provide capabilities that go beyond traditional monitoring techniques.

To help illustrate the concepts of observability, we will use the Fighting Animals example introduced in [Chapter 8](#).

What Is Observability?

The steps involved in an observability solution are essentially:

1. Instrument production systems and applications to collect observability data.
2. Send this data to an external system that can store it.

3. Provide analysis tools that allow the extraction of insights into system behavior for DevOps, SREs, management, and so on.

It is essential that observability data is sent out of the production system and into an entirely separate observability system that is running on another cluster (which should be on physically separate hardware). This can be seen in [Figure 10-1](#) for the Fighting Animals example.

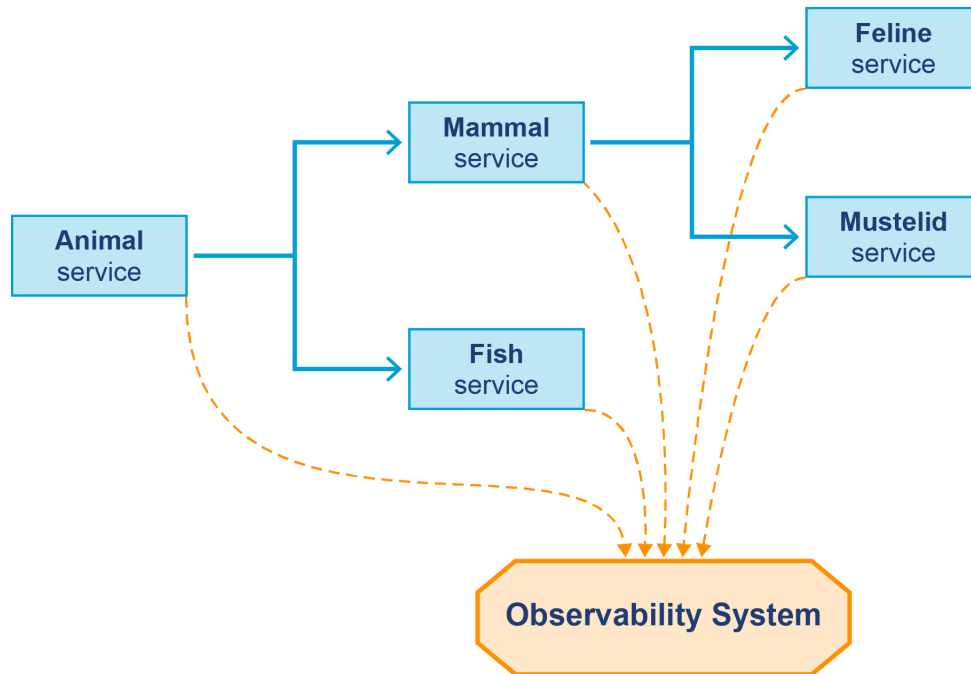


Figure 10-1. Sending observability data to a separate system

The reason for this is hopefully obvious, but to avoid doubt—observability data is used to solve runtime issues and outages. If the data needed to solve the outage is in the same system that is experiencing the outage, it may not be possible to use that data to resolve the outage.

Second, and for the same reason, the observability system must not be upgraded at the same time as the production system under observation.

Third, another extremely important aspect of observability solutions is the need for the analysis tools to be very flexible.

Observability requires that you not have to predefine the questions you will need to ask, or optimize those questions in advance.

—[Charity Majors](#)

An outage analysis based on observability data is exploratory and has aspects that resemble hypothesis testing in data or physical sciences. This means that some sort of graphical representation (including a graph plotter) and a query tool are common user interfaces for observability tools.

Finally, good observability data should provide actionable insights from the entire system, not just single aspects or facets, and this requires sepa-

rate systems, services, and signal types to be correlated or linked in context. This is especially true for larger systems where the volume of observability data can be overwhelming.

Why Observability?

Three threads led to the practice of observability.

First and one of the most important is derived from system control theory, and specifically the question: “How well can the internal state of a system be inferred from outside?” This question naturally arises in incident resolution, but it also has wider utility.

NOTE

One way that observability differs from monitoring is that due to the complexity of modern apps, you can’t really monitor the health of the application without knowledge of the application’s internals. This means adopting some sort of DevOps approach and getting rid of the mentality that “monitoring is the ops team’s responsibility and devs shouldn’t need to care.”

A second thread is the recognition that “cloud is different.” Modern applications often use approaches such as immutable infrastructure as part of the deployment landscape. For example, a container containing the application is built by a continuous deployment system and pushed out to, say, a Kubernetes cluster.

The architectural differences in systems like Kubernetes, as compared to traditional systems, require a new operational approach. These deployments are not intended to be upgraded or to be particularly long-lived. Instead, if there are problems with it, it will be either reverted to a known-good version or rolled forward to a new candidate build.

This immutability of configuration does not work well with traditional system operation, such as SSH’ing into systems to perform interactive debugging and exploration. Observability is intended to deal with this change in emphasis (and scale) of operational practice.

A third and final thread that led to observability is application performance monitoring (APM). This area had originally been dominated by proprietary technology firms such as New Relic, Dynatrace, AppDynamics, etc.

In recent years, new entrants with a more open source approach have appeared, including Datadog and Honeycomb. The overall effect has been that the existing APM vendors have become more general observability-

focused tools and have adopted more open source components as they do so.

The Three Pillars

The *three pillars* are a simple model to explain the primary data sources used in observability analysis. They are represented graphically in [Figure 10-2](#).



Figure 10-2. The three pillars

The pillars refer to data sources that differ in fundamental aspects of shape and form:

- Metrics
- Logs
- Traces

Let's discuss the three pillars in that order.

Metrics

Many Java developers are already familiar with metrics, at least intuitively, but let's be specific about what we mean by a *metric*. Metrics are numbers that measure specific activity over a regular time interval. This means that metrics are often *counters* or *gauges*, and they represent aggregation of a set of data values into a time series.

In this aggregation, the precise detail of every distinct event is lost, but the resulting signal is much more compact, particularly for dashboards/alerts.

NOTE

Metrics can often be used as the starting point of an investigation. Logs and traces can then provide the detailed context that helps you piece together what happened.

There are typically four parts to a metric:

- Timestamp
- Name
- Value
- Dimensions (if any)

These should be fairly self-explanatory, except for possibly the last item—the *dimensions* of the metric.

Dimensions represent different values of some attribute (aka tag), which are recorded as a key/value pair. Crucially, to be suitable for use as a dimension, the metric values must allow for aggregation across dimensions.

As an example, the system and user percentages of CPU utilization can be sensibly aggregated—by adding them together to yield the total CPU utilization. We could represent the metric like this similarly to the common Prometheus format we'll meet later on:

```
cpu_memory_usage{type="system",} 0.12
cpu_memory_usage{type="user",} 0.66
```

This is a well-defined dimensional metric: it's named

`cpu_memory_usage` and has a single dimension (`type`) with only two possible values— `system` and `user` —and the two values for the metric (`0.12` and `0.66`) can be sensibly aggregated into a total value.

On the other hand, the temperatures of separate rooms in your house, say kitchen and bedroom, do not have a particularly useful aggregate. Adding them together makes no sense, and the average is at best marginally useful. In this case, the temperatures are best represented using separate names, rather than dimensions.

Dimensions are also expected to be relatively *low cardinality*—which means there are only a small number of possible values for the dimension.

Additionally, in practice, metrics may well have multiple dimensions, and thus, the true max cardinality of the metric is the product of the sizes of

the value set for each separate dimension. This can impact both query performance and storage volumes—which, in turn, can raise the cost of implementing observability. It is also possible to experience problems visualizing and interpreting high-cardinality metrics. However, the actual observed cardinality may be less, since—in practice—some of the possible combinations may not occur.

NOTE

It's difficult to be precise about what constitutes too much cardinality for a dimension, but at one extreme, if your system has a million distinct users, then the `user_id` is not going to be a suitable choice for a dimension.

Once metrics are generated, they need to be exported to a storage and analytics system, just as for the other observability data types.

A simple but typical metrics observability installation can be seen in [Figure 10-3](#).

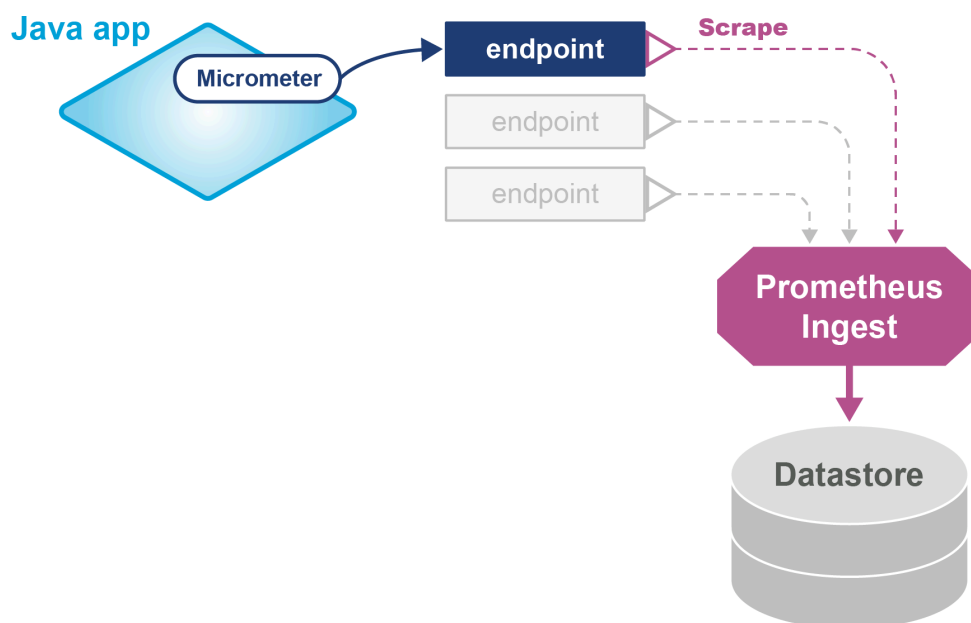


Figure 10-3. Example of metrics observability

As we discussed briefly in [Chapter 8](#), Prometheus is one of the key technologies in the metrics space. In particular, it is probably the most common metrics storage system. We will have a bit more to say about it in the next chapter, but a full discussion of all of its aspects is outside the scope of this book—please consult the product documentation.

Finally, remember that metrics are time series data, i.e., they are reported at a fixed period. The JVM does produce data—for example, garbage collection—which is event-based and does *not* occur at a regular time cadence. These events are not amenable to standard time series analysis, as the technique normally assumes fixed-rate. Therefore, caution is required

when using them as metrics—there are subtleties involved in the data handling.

For example, in the case of GC, to produce metrics, we must aggregate events over a fixed window. These metrics can be useful (especially when represented as histograms), but it's important to remember that information has already been lost compared to the underlying events.

Let's move on to look at the second pillar, logs, which is also likely to be familiar to Java developers.

Logs

Java development has had—and continues to have—a rich tradition of logs and log handling. Many, perhaps even most, Java teams are used to standard logging frameworks (SLF4J, Log4j, etc.).

The use of a logging facade pattern is very common, and to Java developers it's “just a logging API.” In this model, the log configuration defines a suitable log exporter (which may be a file writer, a network logger, or something more elaborate, such as an exporter that sends to JDBC). The developer is agnostic to *how* the logs are handled, as the API is the same regardless of the plumbing, as this code example shows:

```
// SLF4J Code Example from logging_only branch
@RestController
public class AnimalController {
    private static Logger LOGGER = LoggerFactory.getLogger(AnimalController.class);

    // ...

    private String fetchRandomAnimal() throws IOException, InterruptedException {
        var pause = (int) (SERVICES.size() * Math.random());
        LOGGER.info("Pausing for: " + pause);

        // ...
    }
}
```

But what actually is a log? Log entries are best thought of as records of discrete events that happen at a specific point in time. They are usually understood to be immutable after creation.

Examples include system and server logs (e.g., syslog), firewall or network system logs as well as the developer-focused platform and application logs. This type of log is unstructured text and usually has a severity associated with the log event. The encoding of the log is often plain text,

but there's no reason why it couldn't be binary—we consider this an implementation detail.

Logs have the characteristic that they are human-readable text. This brings useful abilities, such as being able to grep for keywords/phrases/regexes as well as graphing counts of specific messages over time.

In addition, log severity provides an easy verbosity control for how much data we want to look at. They can also be helpful as the data source for an audit tool or trail for forensic-style playback, although this depends upon proper design and architecting of the log system, as we will see later in this chapter.

NOTE

There is also the possibility of an *event*, which is a structured, immutable record of a discrete event that does not have a severity (but may have a name describing the class of the event). These are often represented in JSON format to provide a defined structure, and some practitioners refer to them as *structured logs*.

In a typical observability logging pattern, we aggregate logs from individual microservices to a single location over the network. This provides a single, unified dataset of logs that is searchable, filterable, and groupable. However, it is important to note that this is a *centralized* logging pattern—the logs are all in one place, even though they have come from different processes of a distributed system.

Note that aggregating logs from different sources requires us to have some consistency in how we log across those services.

By contrast, true *distributed logging* is much less common and much more technically complex. It is usually only seen when some aspect or constraint of the system prevents centralized logging from being possible—for example, contention with application processes for network or sheer excessive volume of logs that cannot be reduced. This is an edge case, and not one that most app developers will ever find themselves needing to address. Accordingly, we do not discuss distributed logging any further in this book.

When deploying a centralized logging solution as part of observability, there are a wide variety of open source and vendor tools to choose from. One commonly encountered term is the *ELK stack*. This is an acronym that describes a stack comprised of three popular projects:

Logstash

Data processing pipeline for data ingestion (from multiple sources) and transform

Elasticsearch

Data storage for search and analytics

Kibana

GUI component and KQL query language

When deployed, the ELK stack can look similar to the architecture shown in [Figure 10-4](#).

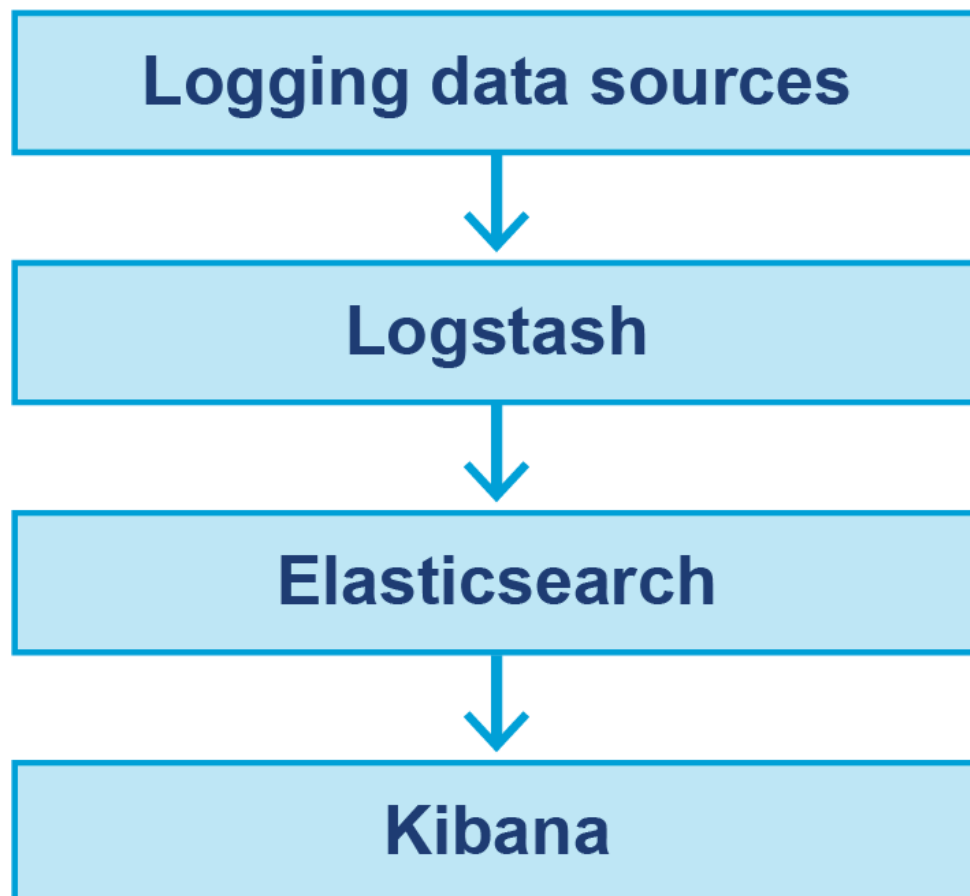


Figure 10-4. Example ELK stack

Running and maintaining an ELK stack can become expensive—not least due to the infrastructure costs involved in hosting it on one of the major cloud platforms. Various companies offer hosted versions of the ELK stack.

In general, DevOps staff are much more likely to be users of an ELK stack through a UI (whether self-hosted or from a vendor), rather than needing detailed knowledge of how to set it up and maintain it.

To complete the pillars, we also need to discuss traces—a subject that Java developers may not be as familiar with.

Traces

A *distributed trace* is a record of a single, top-level service invocation. In the normal case, this corresponds to a single request from an individual user, e.g., triggered by user activity. The trace includes the following metadata about each request:

- Which instance was called
- Which container each subrequest ran on
- Which method was invoked
- How the request performed
- Whether the request was successful or errored
- Optional additional custom metadata to enhance search and retrieval

In distributed architectures, which is the primary case we consider in this book, a single top-level service invocation typically triggers other actions that contribute to the overall trace. These actions are referred to as *spans* and can consist of, for example, a call to a method or a call to a remote service. Each span has the same type of associated metadata as the overall trace, so a trace forms a tree structure of spans.

Note that distributed traces are primarily used to instrument service calls that are request-response oriented, such as HTTP. Calls, such as asynchronous messaging, where there is no response, have additional difficulties associated with them.

In general, distributed tracing is still being developed for asynchronous messaging—but it is actively being worked on by a number of interested parties. As the async story is still in flux, for the rest of the book, we will consider only synchronous, request-response-oriented flows.

An example span might look like this:

```
{
  "name": "/v1/app/foo",
  "context": {
    "trace_id": "kDMI7LTxLxTj220awNARJw==",
    "span_id": "9ir6veJ4Hdw="
  },
  "parent_id": "",
  "kind": 1,
  "start_time": "2021-10-22 16:04:01.209458162 +0000 UTC",
  "end_time": "2021-10-22 16:04:01.209514132 +0000 UTC",
  "status_code": "STATUS_CODE_OK",
  "status_message": "",
  "attributes": {
    "key": "attr",
    "string.value": "value2"
  }
}
```

```
// ... HTTP and transport details omitted
},
"events": [
  {
    "name": "",
    "message": "OK",
    "timestamp": "2021-10-22 16:04:01.209512872 +0000 UTC"
  }
]
}
```

Note the `trace_id`, `span_id`, and `parent_id`—these three fields will be used to connect the different spans and reconstruct the distributed trace, as you will see. In our example, this is a root span, because there is no `parent_id`.

The span perspective corresponds to what is known as the *extrinsic view* of service calls in traditional monitoring. The parts of a single user call (the spans, in other words) are collected into a trace, which can be visualized as a *traceview*. This is a representation of how the tree of spans was generated.

You can see an example in [Figure 10-5](#), which uses the Fighting Animals example again, and is visualized in the Jaeger UI.

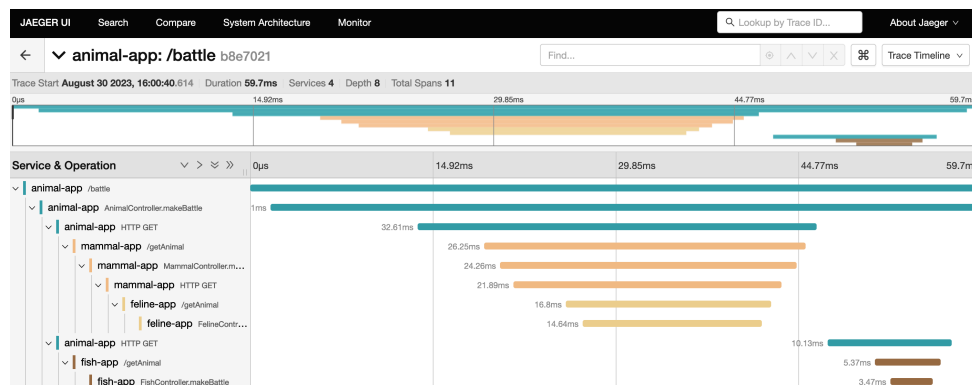


Figure 10-5. Example traceview

To implement distributed tracing, a team needs to consider several aspects of their architecture. The most important are:

- Instrumentation of application, middleware, and other components
- Propagation of trace context between services
- Trace data ingest and storage
- Search and retrieval of traces of interest
- Trace visualization

This can be done using either vendor tools (such as Datadog, New Relic, Dynatrace, Honeycomb, etc.) or by deploying a combination of open source tools. Some of the most common tools and projects used in open

source tracing are OpenTelemetry, Jaeger, and Grafana Tempo. The tools provide different roles and are often combined to produce an overall solution.

For example, in [Figure 10-6](#), you can see OpenTelemetry being used for instrumentation, trace context propagation, and data transport (aka *export* or *exfiltration*) to the observability system. Data storage is provided by Tempo and visualization by Jaeger.

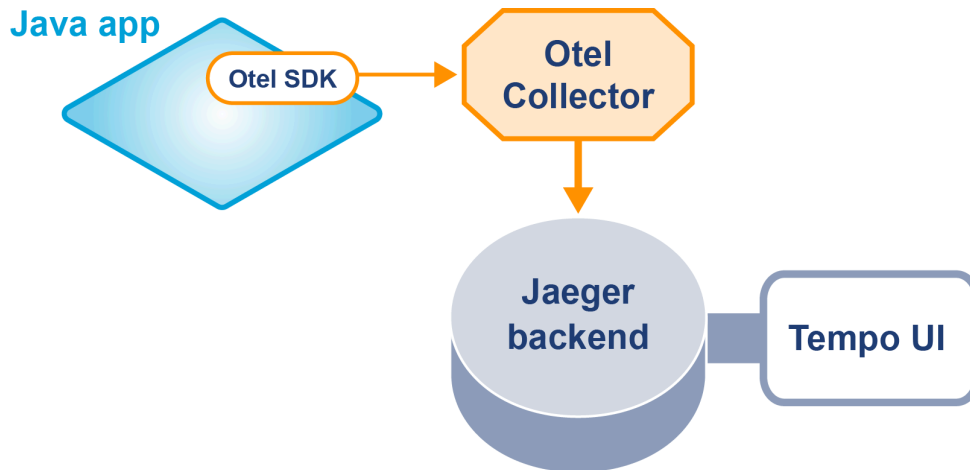


Figure 10-6. Example of distributed tracing observability

The context propagation part is handled by the three fields: `trace_id`, `span_id`, and `parent_id`. These values must be passed from microservice to microservice, and at first glance, it is difficult to see how this can be achieved, given that the services may be in different JVMs or physically separate hosts.

In practice, the values for these fields are carried in additional HTTP headers, as per the [W3C Trace Context format](#), which is a W3C Recommendation as of March 2024. These headers can either be handled manually by application code or via automatic instrumentation and injection. We will see examples of both (for the case of OpenTelemetry traces) in the next chapter.

In fact, visualization of traces is usually the hard part—the traceview is the usual form (e.g., Jaeger). However, this is often too low level and difficult to correlate with a larger overall picture of the system. At time of writing (August 2024), it very much seems as though some new forms of trace visualization are badly needed—especially as systems become more complex over time.

Finally, what about data sampling? Distributed tracing can produce very large volumes of data. Different sampling policies can be used to rationalize the data volumes arising from traces.

In practice, the overwhelming majority of traces will be for requests that return a success code (if not, then you have other problems), and it is not really necessary to handle the entirety of the dataset arising from successes.

NOTE

The W3C specification also defines the `trace-flags` field, which controls tracing flags such as sampling, trace level, etc. as recommendations from the caller to downstream services.

The intent of sampling is to preserve the statistical properties of the dataset. This is important because, for example, any changes in the distribution of response times due to a code change need to be observable.

As an example, the default sampler policy in OpenTelemetry is the “ParentBased Always On” sampler. This policy is: if there is no parent span, sample this span. If there is a parent, sample according to whether the parent was sampled. This essentially delegates the sampling decision to the origin point of the trace.

This has the benefit of simplicity, but has the problem that when the sampling decision is made, the sampler can’t know whether the trace will succeed or have errors.

NOTE

Traces containing failure codes—whether client or server side, i.e., 4XX or 5YY in terms of HTTP—are typically more interesting than successes and ideally should always be collected unsampled, but this requires *tail-based sampling*, which remains challenging.

The sampling approach also explains why both `trace_id` and `parent_span_id` are propagated.

The Pillars as Data Sources

The three pillars approach essentially requires us to treat the signals from each pillar as separate data sources when they’re generated. This is largely because the data sent from each pillar is structured very differently:

- Metrics are four fixed fields (with the dimensional field being a set of key-value pairs) that represent events aggregated over a regularly cadenced time period.

- Logs are unstructured text with a severity (and events are structured data with no severity) that occurred at a specific point in time (i.e., a `Java Instant`).
- Traces are made up of spans, which are defined, structured metadata that occurred over a duration (which can vary from trace to trace).

The data volumes produced also differ wildly from each other:

- Metric volumes don't grow with request traffic, as they are produced at fixed time intervals regardless of how many requests have been received.¹
- Log volumes do not have a simple relationship to request rates. In a system with poor logging discipline (whether in normal operation or when experiencing a lot of errors), the growth can be super-linear.
- Trace volumes are linearly related to request traffic. This remains more or less true when sampling traces.

These differences have caused open source projects that process observability data to tend to focus on only a single pillar. This siloing of projects has consequences for organizations that want to deploy integrated OSS observability.²

The real power of observability is bringing all this data together—the ideal is to be able to correlate across the different data types, not just collect and analyze each individually. Providing correlation between the signals is an active area of development in the open source tools, but at the time of writing (August 2024) this is not yet a solved problem.

Finally, it is worth noting that not every observability practitioner agrees with the three pillars as a basic model. In this book, we have chosen it as our primary approach for two reasons:

- Newcomers are likely to be familiar with the basic data types and the concepts that follow from them.
- The data types as discussed are, as we have seen, fundamentally different from each other.

Other approaches do exist, and you may well encounter them as you progress on their observability journey. As ever, our intent is to offer a starting point and encourage you to explore beyond.

Profiling—A Fourth Pillar?

In addition to the three standard pillars, there is a growing interest in seeing application profiling as a fourth data type and pillar. There are significant differences between profiling data and the other observability data types, though.

Implementing general, large-scale profiling also involves technical challenges. This is largely because profiling (whether of CPU or memory) can have significant overhead. For this reason, two separate approaches have evolved:

On-demand profiling

Activated only when necessary, provides richer dataset

Continuous profiling

Always on, so overhead is reduced as much as possible

There are tradeoffs to both approaches. For example, continuous profiling will usually have lower overhead (because it has to), but it will also have lower data resolution—so it may be harder to spot problems in the collected data.

On the other hand, for on-demand profiling, something (either a human operator or a monitoring agent) needs to be able to turn profiling on and off again. This not only implies a control plane, but it also requires someone/something to notice that there is a problem in the first place.

It also means that it's hard (or impossible) to use for predictive problem analysis—i.e., being able to see an outage coming.

Profiling is the subject of [Chapter 12](#), and we will have more to say about it there.

Observability Architecture Patterns and Antipatterns

We have already discussed the logging facade as a key pattern within observability. In this section, we will discuss some other patterns, especially in the area of metrics. We will finish by looking at a couple of antipatterns that teams should aim to avoid.

Architectural Patterns for Metrics

Metrics is a well-established aspect of observability for Java applications, and as a result, a number of different architectural styles have emerged to support it.

We have already met the concept of dimensionality, but not all metrics systems support dimensions. The key question we need to ask is: “Does the consumer support key/value annotation of the measurement?”

If yes, then the metric library being used is dimensional. If not, the alternative is to combine the information that is carried in the dimension into the metric name as suffixes to a base metric name.

For example, there are two primary conventions for metric naming:

Dotted

Used by OpenTelemetry Metrics and many vendor tools and libraries

Snake-cased names

Used primarily by Prometheus (but Prometheus also has dimensions, which it refers to as *metric labels*)

A dotted convention would look like this:

`jvm.memory.used{area=heap,id=G1 Eden Space}` whereas the same metric in a nondimensional suffixed naming convention could look like this: `jvm_memory_used_heap_G1_Eden_Space`.

A second aspect of the architecture of metrics systems is *aggregation discipline*. This term refers to where the computation to aggregate metrics is performed:

Client-side

Discrete samples processed (e.g., converted to a rate) before being published out of the application

Server-side

Aggregation occurs at the observability server

In this respect, Prometheus also differs from the alternatives—it prefers to aggregate server-side, whereas most other metrics technologies use client-side aggregation.

This issue once again represents a tradeoff: sending every measurement over the network can be expensive, but so can the additional resources required for client-side temporary storage and math.

Finally, let's consider the question of how metrics should be delivered to the observability system. Two fundamentally different approaches are:

Server poll

The observability system collects metrics from the application.

Client push

The application being observed actively sends metrics to the observability system.

Of these, Prometheus uses server poll, which it refers to as *scraping*. This means that every service you wish to monitor using Prometheus must provide a Prometheus metrics endpoint available over HTTP. Prometheus also relies on services being known or discoverable, which presents challenges for short-lived jobs and for systems that can't (or don't want to) expose a metrics endpoint for security reasons.

To handle this, and also to integrate better with technologies like OpenTelemetry, Prometheus also ships a *remote-write* option, but it is not always deployed by teams.

We will discuss this in more detail in [Chapter 11](#).

Manual Versus Automatic Instrumentation

Manual instrumentation involves making changes to application code to add in explicit calls to a telemetry library.

In the case of tracing, this provides full control over when traces and spans are created and finalized. However, it has two major downsides:

- It enforces direct coupling to observability libraries (at least an API).
- There is significant potential for human error—it assumes that the developer already knows what needs to be traced.

The second point is much more significant than many developers realize.

One of the key aspects of observability is that it must enable you to “find answers to questions you didn't know you had at the start.” If you don't know what the questions are going to be, then how can you be sure that you have manually instrumented the subset of call paths that will allow you to ask those questions?

In other words, manual instrumentation of traces runs a very real risk of introducing observability blind spots into your application. This can be hugely harmful, as they can and will give a false impression of where the problems are.

Logging (e.g., SLF4J or Log4j) is almost always done manually, by explicitly including calls to a logging facade. This can lead to cases where necessary log events are missing or are being generated at the wrong severity level. However, compared to missing trace information, this is a minor annoyance, and teams are used to adding additional logging or changing severities in response to an incident.

Therefore, due to the inherent complexity, many teams prefer *automatic instrumentation* for tracing. This is provided either by using a Java agent or the built-in support provided by the framework your application is written in.

In the case of agents, the technique depends upon bytecode weaving. As noted in [“Summary”](#), installing a Java agent allows the tooling component to modify the bytecode of any class loaded during the application’s run. This allows the agent to automate the insertion of what can otherwise be rather tedious boilerplate code.

NOTE

An OpenTelemetry example can be seen in [“Manual Tracing”](#), if you want to read ahead.

Just as we discussed for garbage collection, a correct automatic system is not subject to human fallibility. However, the agent approach may still require some config (i.e., so it doesn’t instrument absolutely everything). In addition, there may be possible startup time impact and runtime performance penalties—at least under some circumstances.

Some of the leading Java frameworks also ship with automatic tracing support—for example, Quarkus can switch on OpenTelemetry just by adding the Maven dependency:

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-opentelemetry</artifactId>
</dependency>
```

There are no mandatory configuration parameters for the extension to work—by default, it will try to send traces to port 4317 on localhost using OTLP over GRPC.³

To complete the picture, metrics typically combine aspects of both manual and automatic instrumentation. Developers can set up their own metrics for the items of interest that are specific to the application, and these are coded manually. These metrics can then be augmented with automatically generated metrics that are produced by libraries (or by the JVM itself). We will have more to say about this hybrid approach in [Chapter 11](#), when we discuss Micrometer.

Antipattern: Shoehorn Data into Metrics

Recall that metrics are counters, gauges, or histograms—they represent data collected and/or aggregated over a fixed time period. Data that does not fit into this conceptual view is not a suitable candidate for a metric.

This also applies to dimensions—if the values of a dimension can't be aggregated, then it's not a dimension, and the values must be treated as separate metrics.

Remember that a metric, by nature, is a compression of a distribution of events into a time series. Information about the shape of the distribution is inevitably, irretrievably lost whenever we do this.

For example, let's suppose we are interested in the number of 500 error codes from one of our services. They obviously arrive at different times, but if we bundle them into a metric `errors.per.hour`, then the scenarios where one request per minute fails, and 60 requests failed in the last 0.1 s of the hour look identical according to this metric.

As a statistic, averages (including the arithmetic mean) have many practical uses. Properly understanding a distribution isn't one of them.

—[Brendan Gregg](#)

In particular, don't aggregate percentiles—it doesn't work. The reason is very simple—to compute a percentile, you need the original data (population).

Without this, you do not have the ability to determine when your averaged percentiles are lying to you. For example, when you have lots of outliers, they could make unhealthy systems appear healthy, and vice versa.

These faulty aggregations can result in incorrect analysis and poor technical/business decisions. It's not only that the aggregations lie—it's worse than that. They are lies clothed as truth. The humans will look at the pretty charts and take them as the true state of the system—when, in fact, they can be completely misleading.

Another critical point is not to try to collect absolutely everything—not everything matters. This antipattern is sometimes seen when a development team is collecting, for example:

- Very detailed runtime-specific numbers
- Metrics that are too coarse-grained (e.g., five-minute CPU average)

The first case can occur when the team is interested in a number that appears to be pertinent to some problem they are solving, and they forget that it is not interesting in the general case. This can happen when a dev team is trying to make a metrics system provide data that should be coming from another source—such as a tracing system or a profiler. For certain types of triage, metrics just aren't the answer, and adding them unnecessarily may detract from the overall usability and health of the observability platform.

The second case is information that may not be actionable—by the time you notice that the five-minute CPU average is much too high, your system is already in trouble, and Kubernetes may have already started killing Pods.

A good rule of thumb is, when considering adding a metric, always ask: “What issue can this metric help us debug?” If you can't answer that question, then you probably don't need the metric. It's also worth checking whether the proposed new metric can be directly derivable from existing metrics—if so, then perhaps it should be synthesized by the metrics observability backend instead.

Antipattern: Abusing Correlated Logs

This is an attempt to somehow avoid deploying a real distributed tracing solution by pushing request metadata into logs instead. It is often characterized as:

“If we have an ID that flows through logs, we can follow requests.”

This seems to be true at first glance, but it only holds true provided that:

- All logging components are set up correctly (e.g., log level).
- The IDs are always created the same way.
- The IDs do not contain PII/other sensitive data.
- The logging system can keep up with 100% of log flow.

In particular, the ID field must not be just a base-64 encoded form of some PII data (an appalling state of affairs that one of the authors has actually seen).

Not only that, but to make the homebrew log-correlator work, you also have to be prepared to:

- Pay for 100% of log storage for as long as you need it.
- Pay for enough compute to crunch 100% of log flow.
- Create and maintain custom code and integrations for every component in the solution.

Antipatterns are, of course, frequently encountered bad solutions to common problems. However, sometimes things just go wrong, even if we built and operated our systems perfectly. Let's move on to look at that—an introduction to what happens when systems just break, and how to use observability to spot the problems.

Diagnosing Application Problems Using Observability

It is very important to understand the most common ways in which an application can break so you can see how you might be able to detect—and fix—these failures using observability tools. So, in this section, we will discuss some of the most common errors that your applications may encounter.

*It is much easier after the event to sort the relevant from the irrelevant signals. After the event, of course, a signal is always crystal clear. We can now see what disaster it was signaling since the disaster has occurred, but before the event it is obscure and pregnant with conflicting meanings.*⁴

—Roberta Wohlstetter

Throughout the section, we'll also document some of the failure modes of distributed systems. There are many others, so this section is intended to serve only as a primer.

Performance Regressions

One of the simplest issues that observability can help detect is a performance regression. Sometimes, a code (or config) change can have unexpected impacts on some aspect of performance. This can be low-level aspects such as CPU utilization, memory usage, or allocation rate, or higher-level numbers such as overall transaction latency.

An observability tool, with a query interface and a graphing component, makes it possible to see the impact of a change by looking at a suitable observable before and after the change. This can be done either by comparing performance before and after, or by using one of the deployment techniques we met in [Chapter 9](#).

An example of the second type can be seen in [Figure 10-7](#), where we are using a canary deploy.

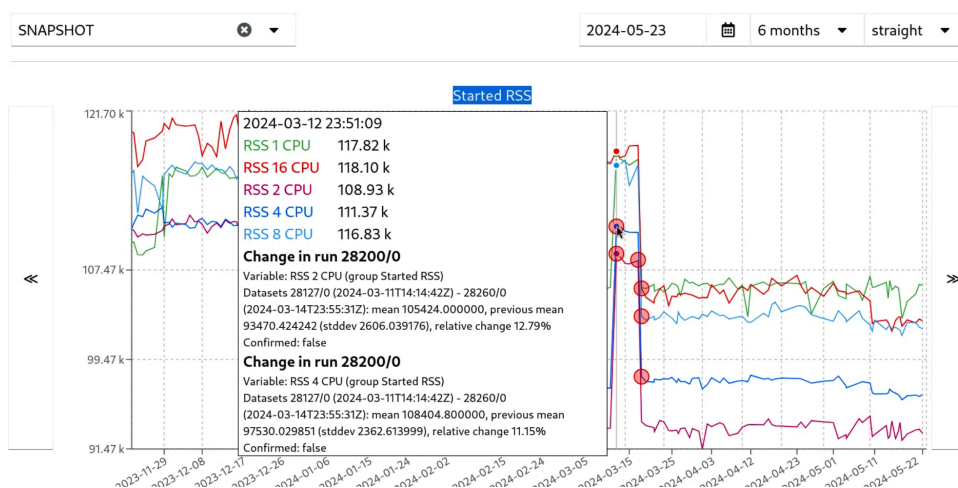


Figure 10-7. Seeing an outlier

Regressions are clear to see with these deployment techniques, because if the modified processes display a substantial divergence from the established baseline in performance behavior, then it should be visible in the relevant observable.

Faced with a straightforward regression like this, the team can decide to either accept the regression (if it's small enough) or to back out the change and do further analysis to modify the change to reduce the impact.

Ideally, the further analysis will take place in a development or testing sandbox. However, certain types of regression do not manifest cleanly outside of the production environment, and in this case, a fix-and-redeploy approach is necessary—which is a natural fit for a technique like canary deploys.

Setting service level objectives (SLOs) with an associated metric is a great way to validate throughout release and day-to-day operations that your system is performing as expected. Alerting or build failures can be set up in the situation where an SLO is no longer met, providing an opportunity to react and resolve the issue. Because performance is not an exact science, error budgets should be applied to set up a tolerance for a given metric. Considering using metrics as SLOs and applying error budgets fits into the SRE discipline and is extremely helpful in maintaining the desired performance of a system.

Unstable Components

Another common problem is a change introducing an instability to an application component. The easy cases of this, for example, when the entire component is destabilized, are often immediately apparent and can be backed out at once. However, it is quite common that the instability manifests only on certain code paths or for certain types of request. In this

case, the change may be completely deployed to production before the component encounters code paths or requests that trigger the instability.

This type of issue is one example of the sort of problem that distributed traces can help solve. The detection methodology could involve comparing error rates of traces before and after a change, and looking at the error traces to see if there is a pattern, such as a particular URL or a particular service that is affected.

In this context, the service can be either one of our own services or some external dependency. It is helpful to have metrics for the error rate of these components, and this also demonstrates the exploratory nature of observability work. Digging into the problem can lead to investigating aspects of the system that were not obviously affected.

A compounding issue is that of *seasonality*. Many systems have code paths that are rarely exercised under normal circumstances, but that become over-represented at specific times (particular days of the week or quarter, or one-off special events). In this case, an instability can lie dormant—as the change that introduced it occurred at a time when the vulnerable code paths were not being called sufficiently often for the problem to manifest.

This means that when analyzing these types of problems, an observability engineer must ensure that they do not fall prey to *recency bias*—automatically assuming that the change that was just made is the cause, merely due to temporal proximity.

Repartitioning and “Split-Brain”

One classic problem in distributed systems is known as “split-brain”: what happens when a cluster suffers a communication failure, such that certain members cannot reach all of the others?

In the worst case, the cluster can fracture into two or more pieces—with each fragment maintaining connectivity within itself but being disconnected from the others. As we will discuss in [Chapter 14](#), when multiple nodes believe they are the Leader, this can lead to *network partition*—an example can be seen in [Figure 10-8](#).

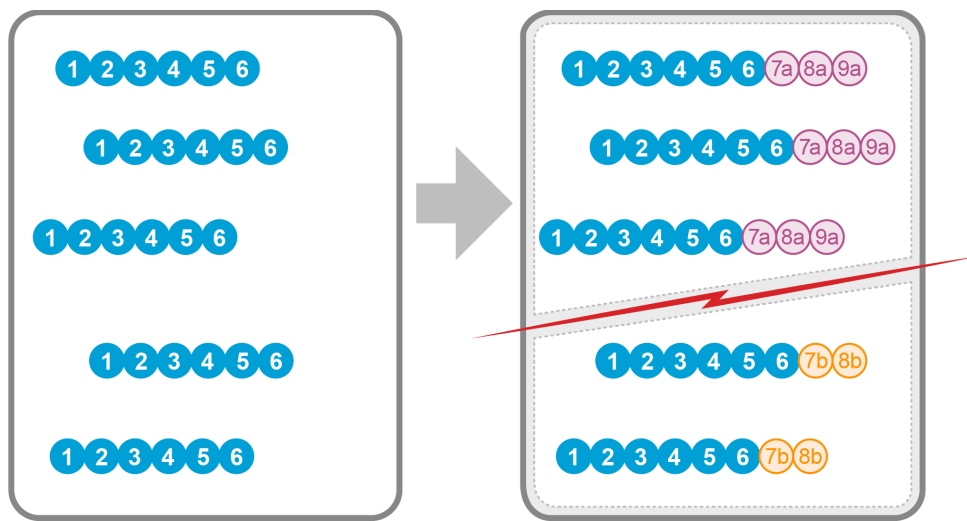


Figure 10-8. Split-brain

This is a significant problem for the observability engineer, as conflicting signals from the fragments can lead to incorrect inference. Even worse is the case where the fragments remain, broadly, in agreement in terms of the system observables.

Can we really tell the difference between a cohesive whole and two fragments that simply happen to be on similar trajectories for now? For this reason, we should ensure that cluster nodes report which node they consider their Leader, and alert if there is ever more than one Leader for a cluster.

NOTE

Network partitioning has been extensively studied, and various techniques for recovering from it, and ensuring data reconciliation, are known. We'll see some more details in [“Basic Distributed Data Structures”](#).

Another related and important question is: if a cluster relies on partitioning techniques, then what happens if a node is lost?

Any system that does not keep multiple copies of the data partitions will lose data in the event of a hard failure (e.g., JVM crash or kernel panic). Therefore, realistic clustering solutions provide recovery and failover capabilities to prevent data loss. In the worst case, there should always be at least one backup of every piece of data, so the loss of a single member will never cause data loss.

However, even in this case, the partitions containing the data will need to be reassigned and balanced between the surviving nodes, as some partitions will now only have a single copy, and this must be corrected. This is referred to as a *repartitioning event*.

In many systems (e.g., Kafka), when a component is in repartitioning state, then no thread can read or process data. This effectively triggers a distributed stop-the-world event while the repartitioning event completes. These STW events are another signal that an observability engineer can, and should, look for and alert upon.

Thundering Herd

A *thundering herd* is a condition caused when some event triggers a large number of clients to make a request on a contented server resource. Typically only one (or a few) of these requests can be serviced (due to contention), which causes the majority of the requests to stall and possibly even time out.

This can often be caused by component restarts in the system—and it may be in a very different part of the system. Consider the restart of a database or database-like system; this results in closing all active connections. At the point of restart, a thundering herd of reconnects to the database results in increased response times, or worse, saturation that knocks over the resource.

Another way of thinking about thundering herds is by analogy with a highly contended Java synchronization lock—when all the threads are awoken, then only one can win the race to acquire the lock, and all others must block once again. The resource consumption caused by a thundering herd can be much more significant than just some unnecessary thread wakeups, however.

A common variation of this problem involves a cache—several readers attempt to read from the cache, fail, and therefore try to load it from the backing database. This causes huge, unnecessary database load, as each reading simultaneously executes the same (potentially expensive) query against the database.

Thundering herd events often show up as a sudden, unexpected burst of database or backend cache load, which is uncorrelated with incoming user requests, and this can be a useful signal to start from when diagnosing.

To help detect thundering herds, we can use metrics such as active connections. In particular, the derivative (rate of change) of that metric is a good way to detect the sudden ramp-up that corresponds to thundering herd.

In addition, a related popular mitigation strategy is to use a request queue to buffer incoming requests before sending them to the contended

backend resource. If the request queue is some sort of `BoundedQueue`, then this technique forces senders to block, effectively throttling the number of requests, and applying *backpressure*.

Cascading Failure

In its simplest form, a *cascading failure* is when a single failing component of an application causes other subsystems to start to fail as a result of positive feedback.

For example, a single copy of a service can start to fail due to higher-than-usual load. This increases the number of in-flight requests, which increases load on the other copies in the cluster. In turn, this increases their failure probability, which can cause a *domino effect* that eventually takes down the entire service cluster, as can be seen in [Figure 10-9](#).

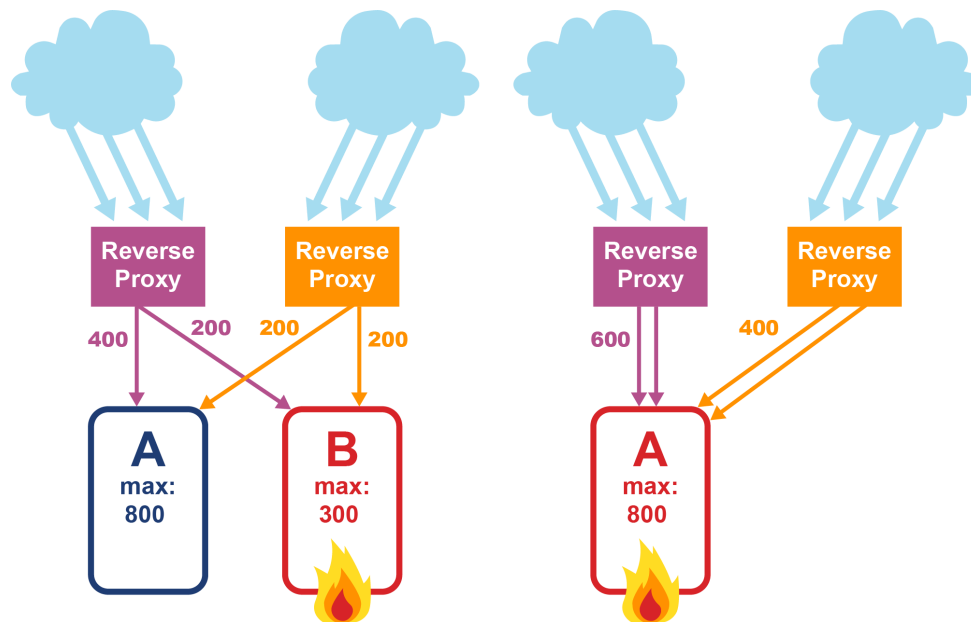


Figure 10-9. Cascading failure

The breakdown and reduction of useful work being done can spread beyond the initially impacted cluster and can potentially spread throughout the system.

For example, the system load balancer may respond to a crashed cluster by routing requests to other clusters. However, if the system is already close to its capacity limits, this can overload the servers in the new cluster, causing further contagion of the breakdown and ultimately leading to a service-wide failure.

The software version of this problem is named after a similar effect in cases such as high-voltage power systems. In that case, a single failure on a fully loaded system results in a sudden spike across all remaining nodes as the electricity is redistributed. This causes other nodes to go into overload and possibly also begin to fail, which makes the problem worse.

However, it is important to realize the limits of the metaphor and similar naming. In particular, physical power systems display two negative aspects that software does not have to deal with when handling a cascade.

Firstly, electrical systems can suffer permanent and irreversible damage if pushed to overload. Software may require some cooldown time, but that's about it—code does not permanently stop working if overloaded once.

Secondly, in electrical systems, the control signals—which would be needed to isolate or shut down other parts of the network to avoid damage—propagate at the same speed as the damaging load, so cannot outrun the cascade. In our case, a well-defined control plane will continue to function even if the data plane is swamped, which means that load balancers, routers, etc. can still be instructed to shed traffic to allow affected subsystems to recover.

This example illustrates just how essential it is that the control and observability infrastructure is distinct from the data plane. If observability fails, it becomes much more difficult (or outright impossible) to know what action to take in the control plane to limit and repair the cascading failure.

Compound Failures

In real outages, some of the failure modes just described can combine with each other to form new and interesting ways for an application to break.

For example, a component can become unstable due to increased downstream latency, which causes the upstream component's requests to sporadically time out.

This failure can be difficult to spot because it can manifest when nothing has changed in the upstream component. An even worse variant is when the only recent changes to upstream were cosmetic or unrelated and the important regression was introduced by a change to downstream (possibly made by a different team) but not immediately seen due to seasonality in the upstream component.

This is a good example of a problem that can be diagnosed with tracing (and that may be more difficult to spot with just metrics and logs), as the increased downstream latency can be correlated with the timeouts of the upstream component.

To take a second example, let's consider increased utilization causing extended processing time, causing request congestion leading to a long GC

pause, which, in turn, causes healthchecks to begin to fail, so K8s recycles the Pod, which causes a Kafka repartition, which causes a cascading failure.

This sort of “perfect storm” failure can be difficult to work out, even with proper observability in place. In practice, an organization’s observability practices will evolve over time—what to observe, what to alert on, and how to react to signals are as much a process as a fixed procedure.

Successful DevOps organizations will conduct retrospectives after these sorts of failures, during which the data collected and actions taken should be reviewed and critiqued. These should, of course, be seen as opportunities to improve observability and should be as blameless as possible.

Vendor or OSS Solution?

There are multiple different possible architectural approaches to deploying observability. Each has its own set of implications for both applications and observability components.

Broadly, the most common choices divide into an initial choice between build or buy—whether to maintain the capability in-house by deploying open source (or possibly on-premises proprietary) components or to pay money to a vendor, effectively outsourcing the observability function.

Organizations that choose to go the route of building their own capability (probably by combining existing open source components) will need to confront the simple fact that the three pillars represent very different data types with different shapes and behaviors. These differences make it very difficult to have a single software tool capable of processing and storing all of these data types—and in fact, there is, as yet, no credible single OSS project that covers all observability needs.

TIP

The OpenTelemetry project, which we will discuss in [Chapter 11](#), is the project that is closest to covering all aspects of observability, but its scope is limited to application instrumentation and signal exfiltration.

This means that organizations that want to use an open source stack that they will maintain in-house will need to make choices about the technologies they want to deploy (and support).

Broadly speaking, open source solutions fall into one of two options:

- Roll your own SaaS pipeline.
- Keep everything on cluster.

The first of these, building your own pipeline based on OSS components, is illustrated in [Figure 10-10](#).

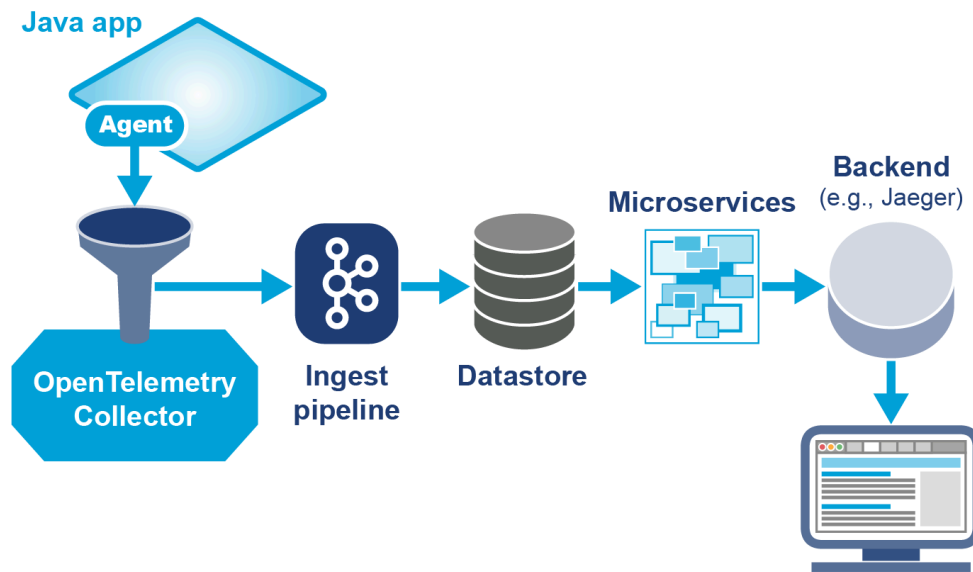


Figure 10-10. Example of an open source pipeline

The second alternative, keeping everything on a single cluster, is closely tied to orchestration techniques—predominantly K8s—and there are a number of different options, including solutions based on eBPF. They all share the drawback that they are single-cluster solutions and can make it harder to spot problems that span clusters.

Turning to the vendor solutions, commercial proprietary products may provide a more integrated experience, including components for which there is currently no viable OSS offering. Examples of vendor-specific components include:

- Alerting
- Serverless monitoring
- Synthetic monitoring
- Real user monitoring
- Outlier/anomaly detection
- Event correlation

Of these, the last is probably the one that most urgently needs to be addressed in the OSS approach, as it is used to tie signals from the three pillars together.

Different vendors offer different products, but two very common architectures for vendor solutions are:

- Aggregate observability data on app host and send aggregates to vendor SaaS

- Send all data to SaaS

The first option is the usual case, due to bandwidth and charging constraints (typically, vendors today use consumption-based pricing).

However, some vendors also offer the second option, but this is often feasible only within a cloud region due to the costs of egress bandwidth charged by the hyperscaler cloud providers. Observability tools can produce a lot of data, so crossing region boundaries can result in large bandwidth bills.

Ultimately, each organization will have to make its own architectural decisions—especially the choice between OSS versus vendor. In larger companies or organizations, there might not be a single outcome. Different business units, different types of applications, different roles (debugging versus prod monitoring) may choose different solutions.

Although vendor solutions represent a direct monetary cost to run, the true cost of implementing and running an observability system (which will naturally be a critical system itself), including the staffing costs, should not be underestimated.

Summary

Observability is emerging as an absolutely essential discipline for cloud native applications.

In this chapter, we introduced you to the basic concepts behind observability. We talked about the three pillars model, which describes the main types of data sources used in observability analysis. We also discussed architecture and met some common observability patterns and antipatterns, including the important question of the choice between in-house and vendor solutions.

One of our key motivations in this chapter was to explain *what* observability can help you diagnose—the actionable insights that should be at the heart of the discipline.

In our next chapter, we will build on these foundations and explain in more detail how to implement an observability solution for a production Java application.

¹ There are some subtleties here around exactly what happens when the mix of incoming requests includes many different dimension values.

² One of the selling points of vendor solutions is this type of correlation.

³ The default changed in upstream OpenTelemetry in early 2024 to be port 4318 using HTTPS/protobuf.

⁴ Roberta Wohlstetter, *Pearl Harbor: Warning and Decision* (Stanford University Press, 1962).